Understanding REST APIs

A Beginner's Guide for Engineering Students

Python + Flask Workshop • Day 3

1. What is an API?

API stands for Application Programming Interface. Think of it as a waiter in a restaurant.

> **Restaurant Analogy**
> You (the customer) don't go into the kitchen to cook your food. You tell the waiter what you want. The waiter goes to the kitchen, gets your food, and brings it back to you. The waiter is the API — a messenger between you (the client) and the kitchen (the server/database).

In software, an API lets two applications talk to each other. For example:

- A mobile app talks to a server to fetch your profile.
- A website talks to a payment gateway to process your order.
- Your Flask app talks to a MySQL database to store student records.

2. What is a REST API?

REST stands for Representational State Transfer. A REST API is a specific style of building APIs that follows a set of simple rules. It is the most popular way to build APIs on the web today.

Rules of REST (Keep It Simple)

Rule	What it means
Client-Server	The frontend (client) and backend (server) are separate. They talk only through the API.
Stateless	Each request is independent. The server does not remember previous requests.
Uniform Interface	All resources are accessed using standard HTTP methods (GET, POST, PUT, DELETE).
Resource-Based	Everything is a 'resource' — e.g. /students, /marks. Resources are nouns, not verbs.

3. How Does a REST API Work?

Every REST API interaction follows this simple flow:

Client (Browser / App) → **HTTP Request (GET /students)** → **Server (Flask App)** → **Database (MySQL)** ← **JSON Response**

Step by step:

1. The client sends an HTTP Request to a URL (called an endpoint).
2. The Flask server receives the request, processes it, and queries MySQL.
3. The server sends back an HTTP Response (usually as JSON data).
4. The client reads the response and displays it to the user.

4. HTTP Methods (The Verbs)

REST APIs use HTTP methods to define what action you want to perform. Think of them as verbs — they describe what you are doing with a resource.

Method	Endpoint Example	What it does
GET	`/students`	Fetch / Read data. Does NOT change anything.
POST	`/students`	Create / Add new data. Sends data in the body.
PUT	`/students/1`	Update existing data. Replace with new values.
PATCH	`/students/1`	Partially update. Change only specific fields.
DELETE	`/students/1`	Delete a resource permanently.

Remember
The URL identifies WHAT resource you are working on. The HTTP Method tells HOW you want to work with it.

5. URL Structure — Endpoints

An endpoint is the URL where your API can be accessed. REST endpoints are resource-based (use nouns, not verbs).

Good vs Bad URL Design
GOOD (noun-based): GET /students/1 BAD (verb-based): GET /getStudent?id=1 Always use nouns for resources. Let the HTTP method define the action.

Common URL Patterns

Method	URL	Action
GET	`/students`	Get list of all students
GET	`/students/1`	Get one student with ID = 1
POST	`/students`	Add a new student
PUT	`/students/1`	Update student with ID = 1
DELETE	`/students/1`	Delete student with ID = 1
POST	`/students/1/marks`	Add marks for student ID = 1
GET	`/students/topper`	Get student with highest marks

6. Request and Response

HTTP Request Structure

Every API request has these parts:

Part	Description
Method	The action — GET, POST, PUT, DELETE
URL	The endpoint — where to send the request (e.g. http://localhost:5000/students)
Headers	Extra info like Content-Type: application/json (tells server the data format)
Body	Data sent with POST/PUT requests. Written in JSON format. Not used in GET/DELETE.

Sample POST Request Body (JSON)

```
{
  "name":       "Rahul Sharma",
  "roll_no":    "CS101",
  "department": "CSE",
  "email":      "rahul@example.com"
}
```

HTTP Response Structure

The server sends back a response with two key parts:

- Status Code — a number that tells you if it worked or not.
- Body — the actual data (usually in JSON format).

Sample JSON Response

```
{
  "message":    "Student added successfully",
  "student_id": 1
}
```

7. HTTP Status Codes

Status codes are 3-digit numbers the server sends back to tell the client what happened. They are grouped into 5 categories.

2xx — Success

Code	Name	When to use it
200	**OK**	Request was successful. Used for GET, PUT, DELETE responses.
201	**Created**	New resource was created successfully. Used after POST.
204	**No Content**	Request succeeded but no data to return (e.g. DELETE).

4xx — Client Errors (your mistake)

Code	Name	When to use it
400	**Bad Request**	Request has invalid data (e.g. missing required field).
401	**Unauthorized**	User is not logged in / no valid token provided.
403	**Forbidden**	User is logged in but does not have permission.
404	**Not Found**	The requested resource does not exist (e.g. wrong student ID).
409	**Conflict**	Duplicate data — e.g. student roll_no already exists.

5xx — Server Errors (server's mistake)

Code	Name	When to use it
500	**Internal Server Error**	Unexpected error in the server code (e.g. DB connection failed).
503	**Service Unavailable**	Server is temporarily down or overloaded.

8. JSON — The Language of REST APIs

JSON (JavaScript Object Notation) is the standard data format used in REST APIs. It is easy to read for both humans and machines.

JSON Rules

- Data is in key-value pairs: "name": "Rahul"
- Keys must be strings (in double quotes).
- Values can be: string, number, boolean, array, object, or null.
- Pairs are separated by commas.
- Objects use { } and arrays use [ ].

Example JSON

```
{
  "student": {
    "id":          1,
    "name":        "Rahul Sharma",
    "roll_no":     "CS101",
    "is_active":   true,
    "subjects":    ["Python", "SQL", "OS"],
    "average":     84.33
  }
}
```

9. Testing REST APIs with Postman

Postman is a free tool used to test REST APIs without building a frontend. Think of it as a way to manually send HTTP requests and see the response.

Steps to test any API in Postman

5. Open Postman and click "New Request".
6. Select the HTTP Method (GET / POST / PUT / DELETE) from the dropdown.

7. Enter the URL: e.g. http://localhost:5000/students
8. For POST and PUT: Go to Body → select raw → choose JSON. Paste your JSON body.
9. Go to Headers → add: Key = Content-Type, Value = application/json.
10. Click Send. Check the Status Code and Response Body at the bottom.

Common Mistake
Always set Content-Type: application/json in the Headers tab when sending POST or PUT requests. Without it, Flask cannot read the JSON body and will return an error.

10. Quick Reference Cheat Sheet

Term	Meaning
API	A way for two applications to communicate with each other.
REST	A set of rules for building APIs using HTTP.
Endpoint	A specific URL where an API can be accessed. e.g. /students
HTTP Method	The action — GET (read), POST (create), PUT (update), DELETE (remove).
Request	The message sent by the client to the server.
Response	The message sent by the server back to the client.
JSON	The data format used to send/receive data in REST APIs.
Status Code	A 3-digit number in the response telling what happened. 200=OK, 404=Not Found.
Header	Extra metadata sent with a request. e.g. Content-Type: application/json.
Body	The actual data sent in the request (used in POST and PUT).
Base URL	The root address of the server. e.g. http://localhost:5000
Postman	A tool to test REST APIs without writing any frontend code.

You are now ready to build and consume REST APIs!

Happy Coding 🚀